

Design Philosophy: Flexible, Non-Linear Story Authoring

"Structure should serve the writer, not constrain them. The tool must support non-linear thinking without forcing a single path."

Table of Contents

1. [Terminology](#) — What each term means and why we use it
 2. [Model Hierarchy](#) — How entities relate (a flexible graph, not a rigid tree)
 3. [Workflow](#) — Practical usage for Games vs Creative Writing
-

1. Terminology

Story

The top-level container for any narrative project. Represents the entire work as a single entity with metadata but no enforced structure beyond what the author chooses to add.

Properties: `title`, `genre`, `theme_tags`, `language`, `created_at`

Sequence (Act / Chapter / Theme Group)

A **bucket** for organizing related content. The term "sequence" is intentionally generic — it can mean an act in a game, a chapter in a novel, or any thematic grouping the author wants. Sequences are purely organizational; they exist to help the writer think in groups, not to enforce a flowchart.

Properties: `title`, `summary`, `order` (optional), `parent_sequence` (for nested sequences)

Key insight: A sequence has no intrinsic relationship to what's inside it. It can hold scenes, beats, notes, or anything else — and the author changes it whenever they want without breaking structure.

Scene

The **unit of work**. A scene is a discrete chunk of narrative that can be written, edited, and expanded independently. Scenes are the primary entry point for most writers because they represent "one thing I need to get done today."

Properties: `title`, `summary`, `word_count`, `status` (draft/revision/final), `attached_media`, `related_beats` (many-to-many links)

Storybeat

A **key moment or turning point** within a scene. Beats are smaller than scenes — they don't need to be full paragraphs, just meaningful narrative points that drive the story forward. Beats can exist independently of scenes (a "flash idea" beat that may or may not make it into the final text).

Properties: `title`, `description`, `type` (inciting incident / revelation / climax / etc.), `attached_to_scene` (optional, many-to-many), `timestamp_order` (for games only)

Outline

A **hierarchical plan or beat sheet** attached to any level of content. An outline is NOT a rigid template — it's a flexible tool for planning that can be as detailed or as sparse as the author wants. Outlines are optional and never mandatory.

Properties: `title`, `content` (free-form text), `attached_to` (story/sequence/scene), `version`

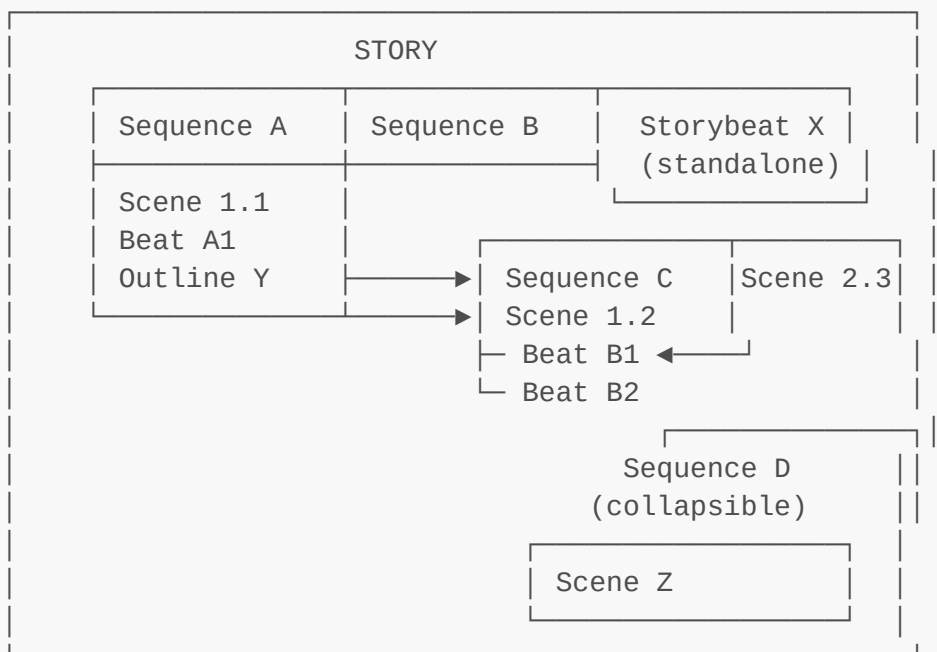
Link (Relationship)

Any connection between entities is a **link**, not an inheritance. A scene can be "related to" a beat, or vice versa — there is no enforced parent-child direction. Links are bi-directional and can be created from any entry point.

Types of links: `references`, `contradicts`, `expands_on`, `summarizes`, `is_part_of` (the only one that implies grouping)

2. Model Hierarchy

The Core Principle: Graph, Not Tree



All arrows are bidirectional. No forced parent/child rules.

Key Design Decisions

Decision	Rationale
No mandatory hierarchy	Writers don't write in a single direction. Some start with scenes, some with beats, some with outlines. All should be equally valid entry points.
Many-to-many relationships	A beat can belong to multiple scenes (a reused motif). A scene can reference multiple beats. This mirrors how writers actually think.
Sequences are optional containers	You don't need a sequence to have content. Empty sequences exist only as organizational buckets the author chooses to use.
Outlines are attachments, not parents	An outline attached to a scene is just notes about that scene — it doesn't "own" the scenes inside it. Delete the outline and the scenes remain intact.

Data Model Summary

► C# Entity description

Why This Matters for ADHD Writers/Gamers

1. **No "wrong" starting point** — If a player thinks in systems, they can create sequences first. If a writer thinks in images, they can write scenes first. Both are valid.
2. **No broken structure when editing** — Moving a scene from one sequence to another doesn't require reordering an outline or deleting old links. The graph handles it naturally.
3. **Zoom flexibility** — You can view the project at any level of detail without losing context. Collapse everything and you still have your story intact; expand and all relationships are preserved.

3. Workflow

General Principles (Both Domains)

- **Start anywhere.** Pick the entry point that matches your current mental state.
- **Edit links freely.** Adding/removing connections between entities should never feel like "breaking" something — it's just rearranging the graph.
- **Outlines are always optional.** They exist only as scaffolding, never as a gatekeeper.
- **Collapse/expand is always available.** You can retreat to overview mode at any moment without losing your place in detail view.

3A. Game Design Workflow (Simple Action RPG Example)

Project: *"Shadows of Aethelgard"* — an action RPG where the player fights through a corrupt kingdom, uncovers a conspiracy involving ancient magic, and ultimately chooses between saving a person or saving the world.

Entry Point: Sequence First (Systems-Thinking Approach)

STORY: Shadows of Aethelgard
 Genre: Action RPG | Fantasy | Political Thriller

Sequence I – The Awakening (Chapters 1-3)

- └ Scene 0.1: Protagonist awakens in ruined village
 - └ Beat: "The door opens, darkness behind you"
 - └ Beat: "A child's toy glows on the floor"
 - └ Outline (attached):
 - Intro scene. Establish tone.
 - Show, don't tell corruption.
 - End with question: "Where am I?"
- └ Scene 0.2: Protagonist finds a weapon
 - └ Beat: "Weapon hums with recognition"
 - └ Beat: "First combat encounter – tutorial fight against corrupted guard"
- └ Scene 0.3: Protagonist learns the truth

Sequence II – The Investigation (Chapters 4-6)

- └ Scene 1.1: Visit to old mentor
- └ Scene 1.2: Discovery of hidden documents
- └ Scene 1.3: First major confrontation

Sequence III – The Choice (Chapters 7-9)

- └ Scene 2.1: Confrontation with antagonist
- └ Sequence IV – Resolution (Chapters 10-12)

Standalone Storybeats (not yet placed in any scene):

- The mentor's secret journal contains a date: November 14th – the night everything changed.
- A recurring visual motif: red thread tying objects across different scenes.

Workflow explanation:

1. The designer starts with **Sequences** because they think in acts and pacing beats (common for game designers).
2. Each sequence is a "chunk" they can work on independently.
3. Within each sequence, they create **scenes** as discrete gameplay segments.
4. **Storybeats** are captured anywhere — sometimes as standalone notes ("the red thread motif"), sometimes attached to specific scenes.
5. An **outline** is added to Sequence I only when the designer wants to plan that chunk more carefully. It doesn't constrain what goes in Sequences II–IV.

Key ADHD-friendly feature: If the designer realizes Scene 0.2 should actually go after Scene 1.2 (they discovered a mechanic they want earlier), they simply **move the scene**. No outline needs updating, no sequence structure breaks. The graph handles it.

3B. Creative Writing Workflow (Novel: ~150k words, 12 chapters)

Project: *"The Glass Garden"* — a speculative fiction novel about an architect who discovers a building that exists outside of time. Contains three interconnected storylines: the protagonist's journey, a parallel mystery in 1974, and a third thread involving a forgotten colony on Mars.

Entry Point: Scene First (Discovery-Based Writing)

STORY: The Glass Garden
Genre: Speculative Fiction | Mystery

[No sequences defined yet – writer is still exploring]

Scenes written so far (unordered, as they were discovered):

- Scene 7: The protagonist finds a clock in the basement that runs backward.
- Scene 3: The 1974 timeline – a letter arrives from someone who shouldn't exist.
- Scene 12: The Mars colony chapter ends with a revelation about the protagonist's grandfather.

Standalone storybeats (ideas not yet placed):

- The clock in the basement is actually a mechanism left by someone from Mars, discovered in Chapter 8.
- A recurring image: broken glass that reflects the future instead of the past.

Workflow explanation:

1. The writer starts with **scenes** — they write "whatever I'm thinking about right now." No sequence structure is assumed yet.
2. As more scenes are written, **patterns emerge**. The writer notices:
 - Three distinct threads (present day, 1974, Mars)
 - Certain scenes cluster around a shared theme (e.g., "discovery of time mechanics")
3. They create **sequences** to group related content — but this is purely organizational. Scene 7 might end up in one sequence, and the "Mars colony" scenes in another — no rigid rules enforced.
4. When planning a chapter for editing purposes, they attach an **outline** to that chapter (which could be multiple scenes grouped into a sequence). The outline is just a planning tool — it can be deleted or ignored at any time without consequence.

Key ADHD-friendly feature: If the writer realizes Scene 7 belongs chronologically after Scene 3 but was written first, they simply **reorder**. No "breaking structure" error. The system doesn't care about chronological

order — only the author's current intent matters.

Comparison Summary

Aspect	Action RPG (Game Design)	Novel (Creative Writing)
Typical entry point	Sequence → Scene → Beat	Scene → Sequence (later) → Beat
Sequences are used for	Pacing, act structure, gameplay segments	Thematic grouping, chapter buckets
Outlines are most useful for	Planning level design and encounter flow	Chapter-level planning and pacing
Primary concern	Gameplay loop + narrative integration	Character arc + thematic coherence
How ADHD benefits from flexibility	Can jump between "design a scene" and "plan the whole act" without friction	Can write freely, then organize later — no pressure to plan everything upfront

Final Note: The Philosophy Behind This Design

This CMS is built on one core belief: **structure should be a choice, not a constraint.**

- Traditional tools force you to pick a workflow and stick with it.
- This tool lets you use *multiple* workflows simultaneously because the underlying data model supports non-linear thinking.

The difference between a rigid tree (traditional CMS) and this graph-based approach is:

Rigid Tree	Flexible Graph
You must choose your entry point at setup	You can enter from any node, anytime
Changing structure risks "breaking" links	All relationships are explicit and editable
Outlines constrain what content is valid	Outlines annotate; they never block
Collapsing hides data permanently	Collapsing just changes viewport — data remains accessible

This is not about being "more flexible." It's about matching the tool to how human brains actually work: non-linearly, associatively, and iteratively.